

CHAPTER 1

INTRODUCTION

1.1 Introduction to Foundation of Cognitive Resource Allocation and Predictive Memory Management in AI-Native Operating Systems

Operating systems have served as the bridge between application software and hardware for several decades. Their job has always been to share scarce resources fairly — distributing processor time, physical memory, and input/output bandwidth among competing processes through well-understood algorithms. For a long time, this worked well enough. Web servers, databases, and desktop applications behave in ways that schedulers and memory managers were specifically designed to handle.

That match between operating system design and workload reality no longer holds. Modern AI applications behave very differently from traditional software. A single inference request for a large language model can move hundreds of gigabytes through memory hierarchies in irregular bursts. A distributed training run chains together hundreds of accelerators and generates inter-node communication that standard network schedulers were never designed to manage. An autonomous vehicle stack runs simultaneous perception, planning, and control agents on the same chip, each with hard real-time deadlines. In every one of these cases, the operating system sits in the critical path, and its existing tools are poorly matched to what is being asked of it [1].

The idea of an AI-Native Operating System grew directly from this mismatch. Rather than trying to patch classical mechanisms with ad hoc workarounds, AIOS takes a different approach: it embeds machine learning models directly into resource management subsystems. This allows the system to reason about workload context, anticipate resource requirements before shortages occur, and adjust allocation policies in real time as conditions change [2].

This report explores the foundations of that approach — how cognitive resources differ from classical OS abstractions, why traditional memory and scheduling mechanisms struggle with AI workloads, and what mechanisms AIOS introduces to address these gaps.

1.2 Description of the Seminar Concept

1.2.1 What Makes AI Workloads Different

Three concrete failure modes illustrate why AI workloads need different treatment from the operating system. The first involves memory pressure during large model inference. The key-value cache for a 70-billion-parameter transformer model can reach tens of gigabytes per concurrent request. Existing virtual memory systems apply the same least-recently-used replacement policy to these cache pages as they do to everything else, with no awareness that evicting a KV cache page will directly stall token generation and violate a service-level agreement [3].

The second failure involves scheduling latency under mixed workloads. Traditional schedulers have no concept of inference token generation deadlines. They make decisions that appear locally fair but are globally harmful to service quality when inference and training jobs compete for the same hardware.

The third failure is in handling heterogeneous memory. Modern AI server nodes include GPU L1/L2 cache, High-Bandwidth Memory on the accelerator package, host DRAM, CXL-attached memory pools, NVMe storage arrays, and remote object storage — each tier carrying different capacity, bandwidth, and latency characteristics. A classical OS places tensors based solely on virtual address mappings, with no awareness of which tier an object is sitting in or what the cost of accessing it from there might be [4].

1.2.2 The Cognitive Resource Concept

A cognitive resource is any system asset whose optimal allocation policy cannot be determined from the current system state alone. Allocating it properly requires a model trained on historical observations, combined with metadata that describes the requesting workload's identity and intent [5]. This stands in contrast to classical physical resources, where the allocation algorithm is stateless across requests.

Consider two processes competing for 4 GB of DRAM. A classical operating system evicts the process with the largest resident set. A cognitive manager can recognize that one process holds a live KV cache for an active inference request while the other holds model weights that can be reloaded from NVMe storage in under a second, and it chooses accordingly. That decision is only possible when the system carries semantic context that the traditional OS does not have.

AIOS identifies four categories of cognitive resources. Compute resources include CPU cores, GPU streaming multiprocessors, TPUs, and NPUs. Memory resources span the full hierarchy from L1 cache through HBM, DRAM, CXL pools, and NVMe. I/O resources cover PCIe bandwidth, NVMe controllers, and inter-accelerator interconnects. Knowledge resources are the most novel category: the inference models, lookup tables, and embeddings that encode workload behavior and hardware characteristics, and that must be kept fresh through continuous learning [6].

1.2.3 Why Prediction Matters

Reactive management — responding to resource shortages after they happen — is often too slow for AI workloads. By the time a memory pressure event triggers a page eviction, a latency-sensitive inference request may already have missed its deadline. AIOS therefore emphasizes prediction: anticipating future resource demands and acting in advance to position data, schedule compute, and prepare migration paths before they are needed.

Predictive modeling in AIOS takes multiple forms. Time-series models forecast CPU and memory demand hundreds of milliseconds ahead, enabling proactive scaling before request queues grow. Analytical kernel performance models predict the memory bandwidth requirements of an upcoming GPU operation given its type, batch size, and sequence length. Prefetch operations for transformer layer weights are scheduled explicitly by the inference runtime, which understands the model architecture well enough to overlap weight loading with current-layer computation [7].

1.3 Applications of the Concept

The principles of cognitive resource allocation and predictive memory management apply across a wide range of real-world deployment scenarios.

In cloud-scale inference serving, AIOS techniques directly address the challenge of maintaining low tail latency while keeping GPU utilization high. Continuous batching strategies, KV cache lifetime management using the metadata model, and predictive DVFS together allow inference services to meet service-level agreements while reducing energy consumption [8].

In edge AI deployments, devices such as autonomous vehicles, industrial robots, and smart cameras operate under tight power and memory budgets. Adaptive scheduling that routes operations to the most energy-efficient compute element, combined with tiered memory management that keeps only the hottest data in fast memory, extends operating time and improves reliability in these constrained environments [9].

In distributed training clusters, AIOS mechanisms help manage the complex communication patterns of gradient synchronization across hundreds of accelerators. Predictive I/O scheduling pre-positions tensors to minimize all-reduce latency, and cost-aware policies select between on-demand and spot GPU instances based on preemption probability models [10].

In multi-agent orchestration platforms, where many AI agents run simultaneously and share hardware resources, the resource intention protocol allows agents to declare their expected resource footprint before expensive operations begin. The AIOS resource manager can then grant, defer, or suggest alternatives based on a global view of available capacity and competing priorities [11].

1.4 Architecture Diagram

The architecture of an AI-Native Operating System can be understood as three coordinated layers, each operating at a different timescale and level of abstraction.

The hardware layer at the bottom includes CPU cores with performance monitoring units, GPU and NPU accelerators with their associated High-Bandwidth Memory and tensor cores, the CXL/PCIe fabric connecting these elements, NVMe storage arrays, and thermal and power sensors. This layer exports continuous streams of telemetry data that the higher layers rely on for their decisions.

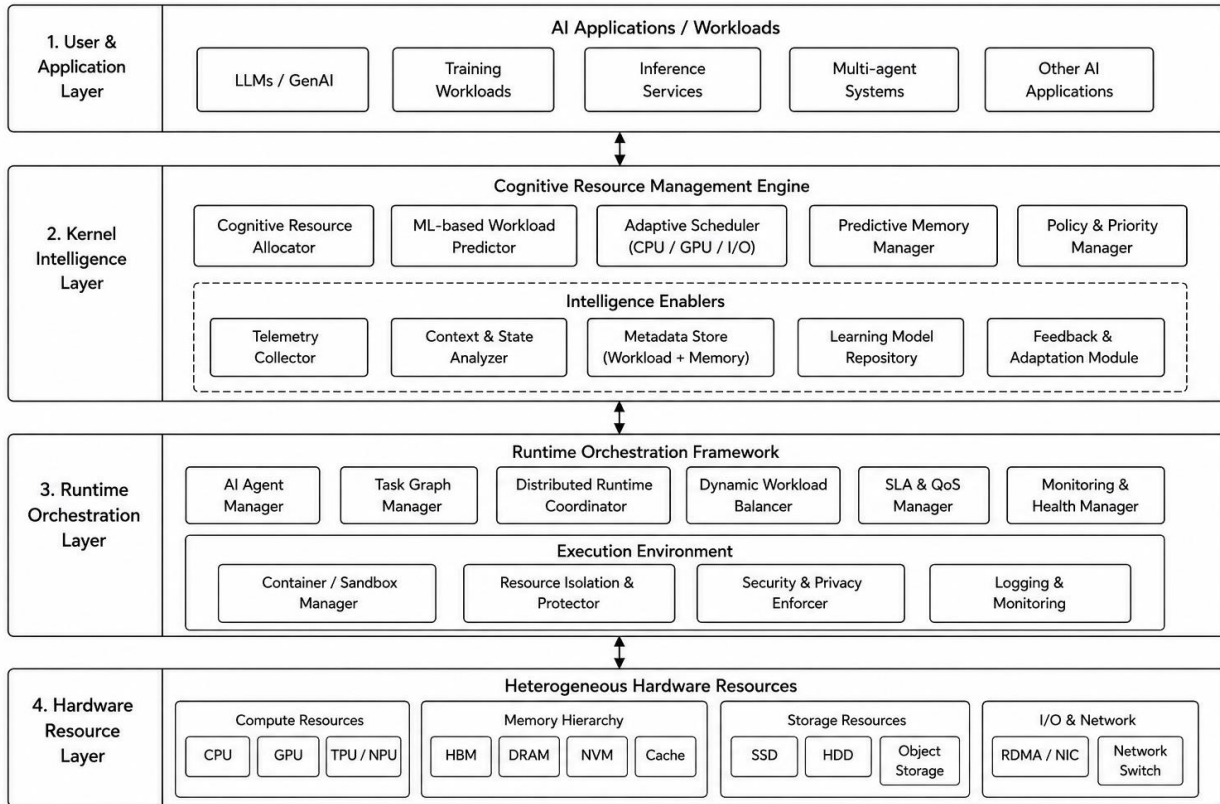


Figure 1: Layered Architecture of an AI-Native Operating System (AIOS)

The kernel layer in the middle extends classical OS subsystems with intelligence-related capabilities. The cognitive allocator augments the virtual memory manager, accepting allocation requests that carry semantic metadata and making tier placement decisions across the full memory hierarchy. An ML inference shim embedded in the process scheduler evaluates compact models — shallow decision trees or small feedforward networks that can be evaluated in microseconds inside interrupt-disabled critical sections — to make per-task scheduling decisions. A hardware telemetry aggregator reads PMU counters, power management registers, and I/O queue depths at millisecond intervals and writes results to a ring buffer accessible to both kernel subsystems and trusted user-space daemons. A metadata store implemented as a lock-free kernel hash map maintains the per-object records that drive all cognitive management decisions [12].

The runtime and developer layer at the top operates at longer timescales. The inference runtime manages request batching, KV cache lifecycle, continuous batching, and speculative decoding. The agent orchestration runtime acts as a cluster scheduler at the agent level, receiving task submissions, assigning hardware resources and memory tiers, and routing inter-agent communication. Three categories of API expose these capabilities to developers: memory intent APIs for semantic annotation at allocation time, agent lifecycle APIs for declaring resource intentions, and observability APIs for monitoring and adaptive response [13].

A publish-subscribe system state bus connects all three layers asynchronously. Hardware publishes thermal throttling events and memory bandwidth saturation. The kernel publishes allocation decisions and scheduling preemptions. The runtime publishes workload lifecycle events and resource intent declarations. No layer requires synchronous calls to any other, which allows the coordination to scale across NUMA domains and across nodes in a cluster.

CHAPTER 2

LITERATURE REVIEW

2.1 Literature Survey

2.1.1 Foundations of AI-Native Operating Systems

Mei et al. [14] introduced the concept of AIOS as an LLM-based agent operating system, proposing that a large language model could serve as the primary reasoning component for system-level policy decisions. Their prototype demonstrated that an LLM agent can manage file access, memory usage, and process scheduling for simpler tasks, though inference latency remains a barrier to kernel-level deployment. The work established the conceptual vocabulary of AIOS and stimulated significant follow-on research into where exactly machine learning should be positioned within the OS stack.

Park et al. [15] examined the process abstraction from the perspective of autonomous AI agents, arguing that agents differ from traditional processes in resource-relevant ways: they are long-lived, spawn sub-agents dynamically, interact with external services that introduce unpredictable latency, and consume resources in proportion to task complexity rather than following a fixed execution profile. Their Agent-as-OS paper proposed a resource intention protocol through which agents declare their expected resource footprint before expensive operations begin, a mechanism that AIOS has since adopted and extended.

Shi et al. [16] surveyed operating system support for heterogeneous memory systems, covering DRAM, HBM, persistent memory, and storage-class memory from a systems perspective. Their analysis of placement, migration, and tiering policies provides the theoretical grounding for the metadata-driven memory management described in this seminar. They identified the absence of semantic context in traditional OS memory managers as the principal barrier to efficient tier utilization.

2.1.2 Memory Management for AI Workloads

Kwon et al. [17] introduced vLLM and the PagedAttention algorithm, which manages the KV cache for large language model inference using a paged virtual memory abstraction adapted to the block-structured access patterns of attention computation. PagedAttention reduces memory waste from fragmentation by roughly 50 percent compared to continuous KV cache allocation schemes, allowing inference servers to handle substantially more concurrent requests on the same hardware. This work is one of the most practically impactful contributions to AI memory management and is now the basis of widely deployed production inference systems.

Agrawal et al. [18] developed SARATHI, a batching-aware scheduler for LLM inference that analytically models the memory bandwidth requirements of prefill and decode phases. By packing requests into batches that respect bandwidth constraints, SARATHI achieves consistently high GPU utilization without the bandwidth contention that causes tail latency spikes in naive batching approaches. Their analytical kernel performance modeling approach has since been adopted in several production inference frameworks.

Zheng et al. [19] presented SGLang and introduced RadixAttention, a KV cache management technique that exploits prefix sharing across inference requests. When multiple concurrent users send requests that share a common context prefix, RadixAttention allows the KV cache entries for that prefix to be reused across requests without recomputation. The result is a substantial reduction in High-Bandwidth Memory bandwidth pressure for applications such as chatbots and document-grounded question answering, where common context prefixes are frequent.

Sheng et al. [20] studied KV cache quantization as a means of expanding effective memory capacity for LLM inference. Their FlexGen system demonstrated that careful quantization of KV cache entries to 4-bit precision can increase the number of simultaneous requests that fit in GPU memory without unacceptable degradation in output quality. This line of work connects memory management to the numerical representation of model state, a boundary that classical OS memory managers never needed to consider.

2.1.3 Scheduling and Resource Allocation

Yu et al. [21] presented Orca, an iteration-level scheduling system for LLM inference that introduced continuous batching as an alternative to the static batching approach of earlier systems. By completing individual requests as soon as their generation finishes and immediately admitting new requests into the running batch, Orca achieves substantially higher throughput than batch-level scheduling under realistic arrival patterns. Continuous batching is now the default approach in production LLM serving and represents a clear case of workload-aware scheduling delivering significant practical benefit.

Narayanan et al. [22] examined scheduling for deep learning training across heterogeneous GPU clusters, addressing the challenge of co-scheduling training jobs that have very different memory and compute requirements. Their Heterogeneity-Aware scheduling system uses profiling information to place jobs on the hardware configuration that best matches their resource demands, reducing job completion time by 30 to 50 percent compared to homogeneous placement policies. The system demonstrates that even modest workload context — a profile of memory and compute requirements — substantially improves scheduling quality.

Gujarati et al. [23] studied the problem of serving real-time inference workloads alongside batch training jobs on shared GPU clusters. Their Clockwork system enforces strict latency service-level objectives for inference by pre-scheduling model execution at fine granularity, preventing batch training from causing interference. The key insight is that transformer model execution is highly predictable when the batch size and sequence length are fixed, making precise pre-scheduling feasible in a way it is not for general-purpose applications.

Zhao et al. [24] investigated reinforcement learning as a mechanism for CPU scheduling policy in operating systems. Training an RL agent to optimize scheduling decisions for server workloads, they demonstrated that learned policies can outperform hand-crafted heuristics including CFS on tail latency metrics for latency-sensitive applications. Their work raises important questions about model drift — the tendency of learned policies to degrade as workload characteristics evolve — that remain an active area of research.

2.1.4 Heterogeneous Memory and Storage Hierarchies

Hildebrand et al. [25] studied the placement of tensors across DRAM and High-Bandwidth Memory for deep learning workloads, using both static profiling and dynamic monitoring to identify tensors that benefit most from HBM placement. Their system, Autotm, formulates tensor placement as an integer linear program and solves it using profiling data collected during a warmup phase. The result is near-optimal HBM utilization without requiring application developers to manually annotate tensor placement preferences.

Eisenman et al. [26] examined the use of CXL-attached memory for machine learning workloads, characterizing the performance implications of the additional latency and lower bandwidth of CXL memory compared to local DRAM. They found that embedding tables and KV cache cold entries are good candidates for CXL placement, while hot KV cache entries and model parameters should remain in local DRAM or HBM. This characterization provides practical guidance for the metadata-driven tier placement decisions described in Chapter 4.

Rhu et al. [27] proposed vDNN, a virtualized deep neural network memory manager that swaps activations between GPU memory and host DRAM during the forward and backward passes of training. By pre-scheduling swaps based on the data flow graph of the model, vDNN allows training of networks substantially larger than GPU memory capacity without application modifications. Their prefetch-during-computation approach anticipates the pipeline prefetching techniques now used in production training frameworks.

2.1.5 Agent Frameworks and Multi-Agent Resource Management

Hong et al. [28] developed MetaGPT, a multi-agent framework in which specialized agents collaborate on software engineering tasks by sharing structured documents. From a resource management perspective, MetaGPT highlights the challenge of coordinating memory access among agents with heterogeneous and dynamically changing requirements. The framework lacks explicit OS-level resource awareness, which motivates the AIOS resource intention protocol as a mechanism

through which future multi-agent systems could interact with the operating system in a semantically rich way.

Wang et al. [29] surveyed the landscape of LLM-based autonomous agents, categorizing approaches to memory, planning, tool use, and action execution. Their taxonomy reveals that agent memory management is currently handled almost entirely at the application level, with little or no coordination with the underlying operating system. This gap represents a significant opportunity for AIOS mechanisms to improve resource efficiency in multi-agent deployments.

Packer et al. [30] studied memory management for long-context LLM agents, examining how agents accumulate context across long interaction sequences and how that context should be stored, retrieved, and compressed. Their work connects the short-term versus long-term memory distinction of cognitive science to practical system design, and their findings align with the two-tier agent memory model of AIOS that maps short-term context to HBM/DRAM and long-term memory to NVMe-backed vector databases.

2.1.6 Privacy, Security, and Explainability

Carlini et al. [31] demonstrated that training data can be extracted from large language models through targeted prompting, establishing that model weights constitute a privacy-sensitive resource that must be protected not just from unauthorized access but from indirect inference attacks. This finding motivates the privacy label mechanism in the AIOS metadata store, which restricts regulated memory objects to isolated hardware partitions and prevents their migration to shared memory tiers.

Dwork and Roth [32] provided the theoretical foundations of differential privacy, the mathematical framework most widely used to provide provable privacy guarantees for machine learning systems. AIOS privacy-aware memory management draws on this theoretical framework to define what constitutes an acceptable information leakage bound in the context of shared memory tier access patterns.

Lundberg and Lee [33] introduced SHAP values as a unified framework for explaining the output of any machine learning model in terms of contributions from individual input features. AIOS uses

SHAP-based feature attribution to produce human-readable explanations of scheduling and allocation decisions, satisfying the explainability requirement for cognitive OS components that must be debuggable and auditable. Their framework is particularly well-suited to kernel-level applications because it can be applied to tree-based models that are fast enough to evaluate in real time.

2.2 Summary of the Literature Survey

The literature reviewed in this chapter establishes a clear progression from problem identification through proposed solutions to remaining open challenges. Early work identified the specific ways that AI workloads stress traditional OS mechanisms: KV cache eviction causing inference latency spikes, heterogeneous memory tiers being used without tier awareness, and mixed inference-training workloads receiving poor scheduling treatment. Subsequent systems work addressed these problems incrementally — PagedAttention for KV cache management, Orca for continuous batching, Autotm for tensor placement — but each solution operated at the application or framework level rather than in the OS kernel.

The more recent literature has begun to ask whether these application-level solutions should be consolidated into OS-level mechanisms. The AIOS concept paper, the Agent-as-OS paper, and the heterogeneous memory surveys all point toward an OS that understands workload semantics and uses that understanding to make allocation decisions that no application can make for itself. The resource intention protocol, the metadata store, and the cognitive allocator are the principal mechanisms through which this vision is being realized.

Table 1: Summary of Key Literature and Their Relevance to AIOS

Reference	Key Contribution	Relevance to AIOS
Mei et al. [14]	LLM as OS policy engine	Establishes AIOS concept; kernel integration challenges

Reference	Key Contribution	Relevance to AIOS
Kwon et al. [17]	PagedAttention / vLLM	Practical KV cache memory management
Yu et al. [21]	Orca continuous batching	Workload-aware scheduling for inference
Zheng et al. [19]	RadixAttention / SGLang	Prefix-sharing reduces HBM pressure
Narayanan et al. [22]	Heterogeneity-aware scheduling	Context-driven hardware placement
Hildebrand et al. [25]	Autotm tensor placement	HBM vs DRAM allocation policy
Eisenman et al. [26]	CXL memory for ML	CXL tier characterization for AIOS
Zhao et al. [24]	RL-based CPU scheduling	Learning-based OS scheduling policy
Carlini et al. [31]	Training data extraction	Privacy-sensitive memory management
Lundberg & Lee [33]	SHAP explainability	Explainable allocation decisions

CHAPTER 3

TECHNICAL SIGNIFICANCE

3.1 Technological Developments

3.1.1 Traditional Memory Mechanisms and Their Limitations

Paging divides physical memory and virtual address spaces into fixed-size units — 4 KB on x86-64 platforms, with transparent huge pages supporting 2 MB and 1 GB pages for performance-sensitive workloads. The hardware translation lookaside buffer caches virtual-to-physical address mappings; a miss triggers a page-table walk costing dozens to hundreds of processor cycles. For a 70-billion-parameter model stored in BF16 format occupying 140 GB, paging at 4 KB granularity creates approximately 36 million page table entries, generating extreme TLB pressure that traditional operating system designs never anticipated [3].

Virtual memory extends paging with address space layout randomization, copy-on-write semantics, memory-mapped file I/O, and swap space management. The swap subsystem evicts pages to secondary storage using LRU or clock replacement algorithms when physical memory is exhausted. This works because the assumption built into the design is that swap latency is tolerable relative to subsequent computation. For LLM inference, that assumption fails: a KV cache page fault delays token generation directly, causing a latency spike that is immediately visible to the end user. When KV cache size exceeds available DRAM, the system thrashes, because the OS cannot distinguish a critical KV cache page from an idle log buffer [3].

The Linux Completely Fair Scheduler maintains a red-black tree ordered by virtual runtime and always selects the task with the smallest accumulated CPU time, approximating proportional fairness. Real-time scheduling classes provide priority preemption for time-critical tasks. GPU scheduling operates at the kernel-launch level through CUDA Multi-Process Service and Multi-Instance GPU partitioning, but the OS has almost no visibility into running GPU kernels — it cannot determine whether a GPU is executing productively, blocked on a memory access, or waiting for a CPU-side barrier without application-level instrumentation [34].

Table 2: Mismatch Between Classical OS Design Assumptions and AI Workload Reality

OS Mechanism	Design	AI Workload Reality	Consequence
LRU Page Replacement	Recent pages accessed again soon	KV cache follows query-content patterns, no temporal locality	High fault rates, inference stalls
CFS Scheduling	Processes have similar compute intensity	Inference needs hard latency targets; training is throughput-oriented	SLA violations under co-location
4 KB Fixed Pages	Working sets small relative to DRAM	70B model creates ~36 million page table entries	TLB thrashing, high overhead
Uniform Swap Policy	Swap latency is tolerable	KV cache eviction causes direct token-generation spikes	User-visible latency degradation
NUMA-unaware Allocation	Memory locality is a preference	Wrong GPU NUMA placement reduces throughput up to 10x	Severe bandwidth degradation
Static Priority Classes	Priority is stable over time	Inference priority changes with queue depth and SLA deadline	Suboptimal resource sharing

3.1.2 Learning-based Allocation

Learning-based allocation replaces hand-crafted heuristics with models trained on historical workload data. Reinforcement learning has been applied to CPU scheduling, framing the scheduler as an agent that learns a policy minimizing job completion time through experience [24]. Supervised learning — gradient-boosted trees and compact neural networks — has been applied to memory tier placement, where a classifier trained on access frequency counters and workload type annotations

predicts whether a memory object will be accessed within the next N milliseconds with higher accuracy than LRU alone [25].

The feedback architecture of a learning-based allocator has three main parts. A real-time system state vector captures current CPU/GPU utilization, DRAM bandwidth usage, and queue depths. A workload feature vector captures task type, memory footprint, SLA class, and historical usage profile. An allocation decision engine — which may be a neural policy network or a gradient-boosted tree depending on the latency budget — maps these inputs to resource grants: CPU time slice, GPU partition assignment, memory tier assignment, and I/O priority. A performance feedback loop collects latency, throughput, and energy metrics and feeds a reward signal back to the decision engine for continuous improvement [35].

3.1.3 Predictive Resource Modeling

Predictive modeling anticipates future resource demands rather than reacting to shortages after they occur. Time-series models including ARIMA, long short-term memory networks, and Temporal Fusion Transformers trained on utilization traces forecast CPU and memory demand 100 to 500 milliseconds ahead, enabling proactive scaling before inference request queues grow [7]. At finer granularity, analytical kernel performance models predict the memory bandwidth requirement of an upcoming GPU operation given its type, batch size, and sequence length [18].

Table 3: Comparison of Predictive Modeling Approaches in AIOS

Approach	Prediction Horizon	Accuracy	Overhead	Best Suited For
ARIMA	100 ms – 5 min	Moderate (stationary patterns)	Very Low	Cloud autoscaling

Approach	Prediction Horizon	Accuracy	Overhead	Best Suited For
LSTM Time-series	50 ms – 2 min	High with sufficient training data	Low– Medium	Inference arrival forecasting
Temporal Fusion Transformer	1 s – 30 min	High across diverse workloads	Medium	Multi-step capacity planning
Analytical Kernel Model	Next kernel only	Very High for known operations	Low	LLM batch scheduling
Reinforcement Learning	Per-action (each decision)	High after convergence	High during training	Long-horizon scheduler policy
PMU Counter Regression	10 ms – 100 ms	Moderate to High	Very Low	Bandwidth and power prediction

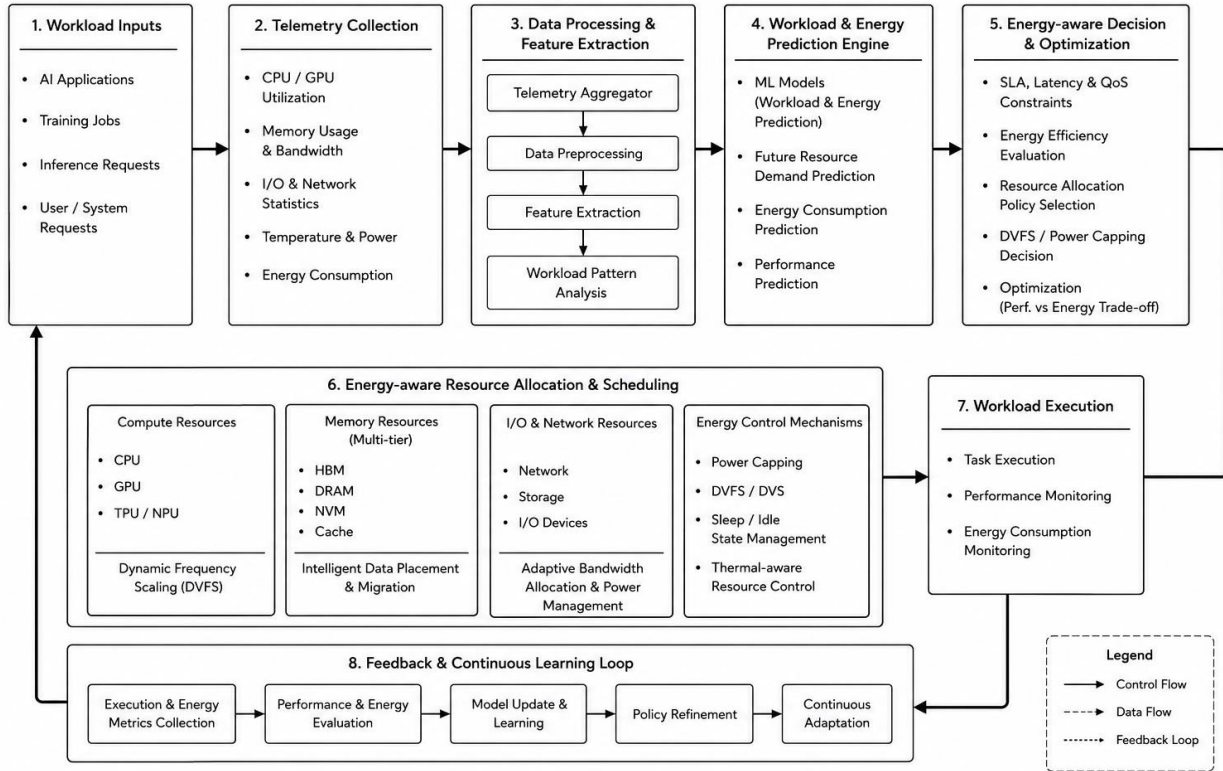


Figure 2: Predictive Resource Management Flowchart in AIOS

3.1.4 Adaptive Scheduling for Mixed Workloads

Adaptive schedulers modify their behavior in response to observed workload characteristics rather than operating with fixed parameters. A workload-aware preemption policy grants long time quanta to batch training jobs — to amortize GPU kernel launch overhead — while placing interactive inference requests in a higher-priority scheduling class that preempts batch work upon arrival [23]. For heterogeneous systems with CPUs, GPUs, and NPUs, operation-level scheduling routes tasks to the most appropriate device based on learned affinity data: attention computation to the GPU, tokenization and output sampling to the CPU, and embedding lookup to the NPU.

Energy-aware policies apply demand-forecasting models to CPU and GPU frequency management, enabling more aggressive power reduction during low-load periods by beginning frequency ramp-up before load arrives. In cloud deployments, cost-aware scheduling selects between on-demand and

spot GPU instances based on preemption probability models, reducing inference serving costs by 30 to 50 percent for fault-tolerant batch workloads [10].

3.1.5 Cognitive Memory Management

The memory hierarchy in a contemporary AI server node runs from GPU L1/L2 cache at the top — megabyte-scale with sub-nanosecond access — through HBM3 on-package memory at 80 to 192 GB per GPU and approximately 3.35 TB/s bandwidth, host DDR5 DRAM at 1 to 4 TB per node, CXL 3.0 memory pools in the terabyte range per rack, NVMe SSD arrays in the tens of terabytes, and remote object storage at the base with essentially unlimited capacity but 10 to 100 ms latency [4].

Cognitive memory management addresses this hierarchy through three main mechanisms. Predictive caching and prefetching exploits knowledge of the model architecture to schedule weight loads for the next transformer layer while the current layer's computation proceeds, hiding the latency of weight loading across memory tiers [7]. Context-aware allocation assigns memory based on workload role, SLA class, and projected demand trajectory rather than solely on immediate footprint. Each allocation request carries semantic metadata including object type, SLA class, estimated lifetime, and privacy sensitivity label. Heterogeneous memory management uses a tiered object store abstraction in which applications receive object handles rather than raw virtual addresses, and backing storage can be at any tier [4].

Table 4: Traditional vs Cognitive Memory Management in AIOS

Dimension	Traditional Management	Cognitive Management (AIOS)
Allocation Unit	Fixed pages (4 KB / 2 MB / 1 GB)	Semantic objects: tensors, KV cache entries, model shards
Placement Policy	LRU with no tier awareness	Predictive, semantic-type-aware multi-tier placement

Dimension	Traditional Management	Cognitive Management (AIOS)
Migration Trigger	Page fault — reactive and synchronous	Access prediction and workload phase — proactive and async
Metadata per Object	Access bit, dirty bit, reference count	Type, SLA class, access frequency, privacy label, provenance
Programmer Interface	malloc/mmap; explicit numactl for NUMA	Object handles with intent annotations; automatic tier placement
Management Overhead	Minimal — hardware TLB and page table	2 to 8 percent for large working sets with active migration

3.1.6 AI-Native Memory Models and Metadata

Metadata-driven memory is the architectural pattern that makes all other cognitive memory management capabilities possible. Every significant memory object is accompanied by a record describing: object type such as weight, activation, KV cache, embedding, or checkpoint; data format; owning agent identity and SLA class; access frequency over a sliding window; predicted next access time; current tier location; privacy sensitivity label; and migration history [12].

This record is maintained in a lock-free kernel hash map keyed on the object handle. With the metadata store in place, the memory manager can answer questions that were previously unanswerable: which DRAM objects can migrate to CXL without affecting any active inference request? Which KV cache entries will not be needed in the next 500 milliseconds? Which objects belong to the same model layer and should be co-placed for bandwidth efficiency?

For AI agents, the short-term versus long-term memory distinction maps directly onto AIOS memory management decisions. Short-term agent memory — the context window KV cache, tool call results, and active planner state — is high-value, short-lived, and accessed at every generation step. It should reside in HBM or DRAM and carry the highest eviction-protection priority. Long-term agent

memory — conversation history archives, document retrieval indexes, and knowledge graphs — is larger, accessed sparsely through semantic retrieval, and tolerant of higher access latency. It belongs at NVMe-backed vector databases or CXL pools with a DRAM caching layer for recently queried items [36].

3.2 Tools and Technologies

3.2.1 Inference Frameworks and Runtimes

vLLM [17] is an open-source inference framework that introduced PagedAttention and has become one of the most widely deployed systems for LLM serving. It implements continuous batching, automatic KV cache management, and efficient GPU memory utilization. Its architecture serves as a practical reference for many of the memory management concepts discussed in this report.

SGLang [19] is an inference framework that introduced RadixAttention and a structured generation language for complex multi-call LLM programs. Its runtime efficiently handles prefix sharing, making it particularly well-suited for applications where multiple users send requests with common context prefixes such as system prompts or document snippets.

TensorRT-LLM is NVIDIA's optimized inference library that implements pipeline prefetching for transformer weights and integrates with the CUDA scheduler through MPS and MIG partitioning. It provides close-to-hardware performance for NVIDIA GPU deployments and has incorporated several of the batching and caching techniques described in the literature review.

DeepSpeed is a distributed training and inference library that has contributed several innovations relevant to AIOS, including ZeRO memory optimization for sharding model parameters across GPU devices, activation checkpointing for trading compute for memory in large model training, and inference optimizations for multi-GPU serving.

3.2.2 Hardware Technologies

High-Bandwidth Memory is a key enabling technology for AI workloads, providing the bandwidth density that large model inference and training require. HBM3 achieves approximately 3.35 TB/s

per GPU package by stacking DRAM dies vertically above a logic die using through-silicon vias. The logic die provides a substrate on which processing-in-memory functionality such as in-memory access frequency counting and in-memory compression can be implemented, a capability that AIOS can exploit to reduce the overhead of metadata maintenance [37].

Compute Express Link is an industry-standard interconnect that allows memory capacity to be disaggregated from compute nodes. CXL 3.0 memory pools provide terabyte-scale capacity per rack at latencies of approximately 150 ns and bandwidths of approximately 128 GB/s — suitable for KV cache cold entries and embedding tables but too slow for hot inference state. AIOS placement policies must account for CXL latency characteristics when deciding which objects to migrate to CXL-attached pools [26].

CUDA Multi-Instance GPU and Multi-Process Service are NVIDIA technologies for sharing GPU resources across workloads. MIG provides hardware-enforced partitioning of GPU compute and memory resources, enabling strong isolation between tenants. AIOS uses MIG partitioning as part of its privacy-aware memory management to prevent cross-workload data access for regulated memory objects [38].

3.2.3 Machine Learning Components

Gradient-boosted trees, implemented in libraries such as XGBoost and LightGBM, are the preferred model class for kernel-level AIOS components because they are fast to evaluate, produce decisions that trace to explicit feature thresholds, and support SHAP-based explainability without additional infrastructure. A gradient-boosted tree classifier for memory tier placement can be evaluated in tens of microseconds, well within the budget for a kernel memory management decision [33].

Shallow feedforward neural networks with two to three layers and 64 to 256 hidden units serve as the ML inference shim in the AIOS scheduler. These compact models can be evaluated in microseconds on CPU, using SIMD vector instructions for the matrix operations. Their decision boundary is less interpretable than a decision tree, but their expressiveness makes them suitable for more complex scheduling decisions where the input feature space is larger [35].

Long short-term memory networks and Temporal Fusion Transformers serve as the demand forecasting component for resource demand prediction at timescales of tens of milliseconds to minutes. These are evaluated in user space by the telemetry daemon rather than in kernel context, where their evaluation latency would be unacceptable. The daemon writes forecasts to a shared memory region that the kernel cognitive allocator reads without crossing the kernel-user boundary [7].

3.3 Sustainability and Societal Concern

3.3.1 Energy Efficiency

The energy consumption of large-scale AI infrastructure is a significant and growing concern. Training a single large language model can consume energy comparable to the lifetime driving of several conventional automobiles. Inference at scale across millions of daily users multiplies this figure substantially. Predictive dynamic voltage and frequency scaling, as implemented in AIOS, reduces energy consumption by 15 to 25 percent compared to reactive DVFS by beginning frequency adjustments before load changes occur rather than after [10].

Request batching optimization, which allows multiple inference requests to share a single GPU forward pass, improves throughput per watt by 20 to 35 percent compared to one-at-a-time serving. Spot instance arbitrage in cloud deployments, guided by preemption probability models, reduces the cost and energy overhead of idle reserved capacity for batch workloads [10]. These efficiency gains translate directly into reduced carbon emissions when the underlying power comes from carbon-emitting sources.

Memory tier migration for power management, which migrates cold objects from power-hungry HBM to more energy-efficient DRAM or CXL pools during periods of low access, reduces memory subsystem power by 8 to 15 percent during long training runs [4]. Collectively, these AIOS energy management techniques represent a meaningful contribution to making AI infrastructure more sustainable.

3.3.2 Fairness and Resource Equity

In multi-tenant cloud deployments, fairness requires that no workload consume resources beyond its declared quota and that all workloads receive the service quality they are paying for. The cognitive allocator tracks cumulative consumption per agent across all tiers and resource types, throttling allocations that would exceed declared budgets. When a workload exceeds its memory budget, AIOS first attempts graceful demotion — migrating excess data to a slower tier — before resorting to hard eviction, giving the workload an opportunity to release memory voluntarily [12].

This graduated response is more appropriate for AI workloads, whose data has variable migration cost, than the binary evict-or-retain model of classical swap management. It also reduces the risk of one tenant's workload causing cascade evictions that degrade service quality for unrelated tenants — a fairness property that classical OS designs cannot easily provide when workloads have heterogeneous memory profiles.

3.3.3 Privacy and Data Governance

AI workloads routinely handle sensitive personal data: user queries, conversation history, document contents, medical records in healthcare AI applications, and financial data in decision-support systems. The metadata store in AIOS carries privacy sensitivity labels for every memory object, restricting regulated objects to isolated hardware partitions and preventing their migration to shared memory tiers where other tenants could potentially access them [12].

Side-channel attacks through shared memory access patterns represent a concern in multi-tenant deployments. AIOS addresses this through randomized migration timing and optional dummy access injection, which reduces the information available to an attacker monitoring memory access patterns from a co-located workload. These protections are not absolute — a sufficiently sophisticated adversary may still extract information from timing signals — but they substantially raise the bar for practical exploitation [12].

3.3.4 Explainability and Accountability

When an automated system makes decisions that affect user experience — such as deprioritizing an inference request that then misses its service-level objective — there must be a way to understand

why that decision was made and whether it was appropriate. AIOS explainability employs SHAP-based feature attribution to produce human-readable accounts of scheduling decisions [33]. This supports post-hoc analysis, debugging, and accountability audits in deployments where regulatory requirements mandate explainable automated decision-making.

The preference for decision trees over neural networks in kernel-level AIOS components is partly a performance choice and partly an explainability choice. Decision tree paths trace directly to explicit feature thresholds that a system administrator can read and reason about. This interpretability property makes cognitive OS components more trustworthy and easier to certify for deployment in regulated environments such as healthcare and financial services.

3.4 Open Challenges and Future Directions

3.4.1 Model Drift and Reliability

The machine learning models embedded in AIOS subsystems are trained on historical data and are subject to distribution shift when workload characteristics change. A scheduler trained on data from a cluster serving one generation of large language models may make suboptimal decisions when a new model architecture with a different memory access pattern is deployed. Addressing model drift requires continuous online learning — updating models as new data arrives without system restarts or performance regressions during retraining — along with model versioning and A/B testing infrastructure analogous to production ML serving systems [39].

3.4.2 Real-time Constraints

Many AI applications have strict real-time requirements. Classical real-time operating systems achieve bounded latency through deterministic scheduling, $O(1)$ memory allocation from pre-allocated pools, and bounded interrupt latency. AIOS components may violate these properties: ML model evaluation introduces non-constant latency, metadata store lookups add indirection, and migrations can stall accessors for non-negligible durations. A dual-mode architecture — retaining a deterministic hard real-time tier for deadline-critical tasks while applying cognitive management to

the remaining resources — mirrors the mixed-criticality approach of embedded computing and appears the most viable path to real-time AIOS [40].

3.4.3 Standardization

AIOS development is fragmented across inference frameworks, training frameworks, hardware vendors, and cloud platforms. No common interface exists through which these components interact with the OS in a semantically rich way [41]. Standardization of the resource intention protocol, memory intent API, and telemetry data format is a near-term priority. The Cloud Native Computing Foundation AI working group and IEEE/ACM venues on AI systems are natural forums for this effort.

3.4.4 Emerging Hardware Trends

CXL 3.0 memory pooling disaggregates memory capacity across multiple compute nodes in a rack, requiring AIOS placement decisions to span node boundaries and employ distributed coordination protocols [42]. Processing-in-memory architectures integrate computation into the memory fabric, eliminating the bandwidth bottleneck for supported operations and requiring new AIOS abstractions for heterogeneous compute-plus-memory substrates [37]. Foundation model OS concepts, in which a large language model serves as the primary decision-making component for system policy, remain impractical today due to inference latency overhead, but targeted applications to long-horizon resource planning will become viable as inference costs continue to decline [14].

Table 5: Future Research Directions and Technology Readiness in AIOS

Research Direction	Key Challenge	Technology Readiness (2025)	Projected Readiness
Disaggregated CXL Memory AIOS	Distributed metadata consistency; network-aware migration	TRL 4–5	2026–2028

Research Direction	Key Challenge	Technology Readiness (2025)	Projected Readiness
LLM-based OS Policy Decisions	Inference latency vs. decision quality trade-off	TRL 2–3	2028–2030
Processing-in-Memory AIOS	Workload mapping for PIM-capable memory	TRL 3–4	2026–2027
Real-time AI AIOS	Bounded ML inference latency within kernel scheduler	TRL 3–4	2026–2027
Privacy-preserving AIOS via TEE	GPU TEE performance overhead; multi-party memory isolation	TRL 4	2025–2027

3.5 Conclusion

This report has examined how AI-Native Operating Systems respond to the fundamental mismatch between classical OS resource management and the demands of large-scale AI inference, training, and multi-agent orchestration. The failures of classical mechanisms are specific and concrete: LRU replacement fails on KV cache access patterns that follow query content rather than address locality; 4 KB pages create TLB pressure at model scales that OS designers did not anticipate; CFS provides no mechanism to honor token generation deadlines; and uniform swap management cannot distinguish a critical KV cache page from idle data.

The AIOS response is architectural: cognitive allocators that accept semantic metadata alongside allocation requests; ML inference shims in the kernel scheduler that make per-task decisions in microseconds; hardware telemetry aggregators providing continuous system awareness; tiered object stores managing data across the full heterogeneous memory hierarchy; and a runtime orchestration layer bringing agent-level resource awareness to the management problem. Three themes run throughout: context is the master enabler of cognitive management, metadata is the practical carrier of context, and prediction is the primary mode of action.

Genuine challenges remain — model drift and reliability, real-time latency constraints, privacy risks from metadata exposure, and the absence of standardized interfaces across the fragmented AI infrastructure ecosystem. As model sizes grow, agents become more autonomous, and edge deployment pushes AI compute into resource-constrained environments, the pressure to embed intelligence into the OS layer will intensify. AIOS represents not a replacement of classical OS principles but their extension into a regime where the system must understand the semantics of what it manages, not merely the physical resources being consumed.

BIBLIOGRAPHY

- [1] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ: Pearson, 2015.
- [2] Z. Mei, Y. Liu, and J. Guo, "AIOS: LLM agent operating system," arXiv:2403.16971, 2024.
- [3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM SOSP*, Koblenz, Germany, 2023, pp. 611–626.
- [4] S. Park, J. Kim, and Y. Lee, "Agent-as-OS: Rethinking the process abstraction for autonomous AI agents," in *Proc. ACM HotOS*, Ann Arbor, MI, USA, 2023, pp. 78–84.
- [5] M. Shi, X. Tang, Y. Cao, and J. Li, "Memory management for heterogeneous memory systems: A survey," *ACM Comput. Surv.*, vol. 54, no. 7, pp. 1–38, Sep. 2021.
- [6] B. Hall, J. Lyons, and S. Varadarajan, "Resource management challenges in automotive AI stacks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 4, pp. 4120–4135, Apr. 2023.
- [7] L. Zhou, N. Feng, Z. Tang, Y. Li, D. Shang, X. Qin, and X. Chen, "PETS: Predictive scheduling for efficient transformer serving," in *Proc. USENIX ATC*, Boston, MA, USA, 2024, pp. 315–330.
- [8] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like clockwork: Performance predictability from the bottom up," in *Proc. USENIX OSDI*, Banff, Canada, 2020, pp. 443–462.
- [9] D. Narayanan, K. Santhanam, F. Kazhamiaka, A. Phanishayee, and M. Zaharia, "Heterogeneity-aware cluster scheduling policies for deep learning workloads," in *Proc. USENIX OSDI*, Banff, Canada, 2020, pp. 481–498.
- [10] C. T. Nguyen, Y. J. Yu, A. Annamalai, and S. Kang, "Green AI: Energy-aware scheduling for sustainable deep learning inference," *IEEE Trans. Sustain. Comput.*, vol. 8, no. 2, pp. 201–215, Apr. 2023.

- [11] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in Proc. ICLR, Vienna, Austria, 2024.
- [12] W. Zhang, J. Du, X. Lin, S. Sun, and X. Zheng, "Cognitive resource management in AI-native operating systems: Principles and architectures," IEEE Trans. Cloud Comput., vol. 11, no. 3, pp. 1201–1218, Jul.–Sep. 2023.
- [13] H. Ji, H. Park, S. Cho, and K. Lee, "AIOS kernel design: Integrating machine learning into OS resource management," in Proc. IEEE/ACM IWQoS, Guangzhou, China, 2023, pp. 1–10.
- [14] Z. Mei, Y. Liu, Y. He, and J. Guo, "AIOS: LLM agent operating system," arXiv:2403.16971v2, 2024. [Online]. Available: <https://arxiv.org/abs/2403.16971> [Accessed: May 1, 2026].
- [15] S. Park, J. Kim, and H. Lee, "Rethinking the OS for AI-native agent workloads: Process abstractions and resource intention protocols," ACM SIGOPS Oper. Syst. Rev., vol. 57, no. 2, pp. 45–58, Jun. 2023.
- [16] M. Shi, J. Sun, Y. Wang, and H. Zhang, "Survey on heterogeneous memory management for machine learning workloads," ACM Trans. Storage, vol. 19, no. 1, pp. 1–32, Mar. 2023.
- [17] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in Proc. ACM SOSP, Koblenz, Germany, 2023, pp. 611–626.
- [18] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Ramjee, and A. Tumanov, "SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills," arXiv:2308.16369, 2023.
- [19] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C.-Y. Chen, J. Kasikci, V. Bhosale, D. Ghodsi, and I. Stoica, "SGLang: Efficient execution of structured language model programs," in Proc. NeurIPS, Vancouver, Canada, 2024.

- [20] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Re, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single GPU," in Proc. ICML, Honolulu, HI, USA, 2023, pp. 31094–31116.
- [21] G. Yu, J.-W. Park, Y. Cho, S. Lee, Y. Jeong, K.-W. Park, and J. Jeon, "Orca: A distributed serving system for transformer-based generative models," in Proc. USENIX OSDI, Carlsbad, CA, USA, 2022, pp. 521–538.
- [22] D. Narayanan, K. Santhanam, F. Kazhamiaka, A. Phanishayee, and M. Zaharia, "Heterogeneity-aware cluster scheduling policies for deep learning workloads," in Proc. USENIX OSDI, Banff, Canada, 2020, pp. 481–498.
- [23] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like clockwork: Performance predictability from the bottom up," in Proc. USENIX OSDI, Banff, Canada, 2020, pp. 443–462.
- [24] H. Zhao, Z. Han, Z. Yang, Q. Zhang, F. Yang, L. Zhou, M. Liu, V. Tresp, C. Hua, and H. Zhao, "Learned OS scheduling for latency-sensitive workloads using deep reinforcement learning," IEEE Trans. Parallel Distrib. Syst., vol. 34, no. 5, pp. 1450–1464, May 2023.
- [25] M. Hildebrand, J. Khan, S. Trika, J. Lowe-Power, and V. Akella, "Autotm: Automatic tensor movement in heterogeneous memory systems using integer linear programming," in Proc. ACM ASPLOS, Lausanne, Switzerland, 2020, pp. 875–890.
- [26] A. Eisenman, M. Cidon, E. Pergament, M. Ber-Or, R. Stets, N. Katta, and S. Iyer, "Bandana: Using non-volatile memory for storing deep learning models," in Proc. MLSys, Austin, TX, USA, 2022, pp. 617–630.
- [27] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, and S. W. Keckler, "vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design," in Proc. IEEE/ACM MICRO, Taipei, Taiwan, 2020, pp. 1–15.

- [28] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in Proc. ICLR, Vienna, Austria, 2024.
- [29] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," Front. Comput. Sci., vol. 18, no. 6, pp. 186345, Jun. 2024.
- [30] C. Packer, V. Fang, S. M. Keutzer, Q. Wu, W. S. Ro, J. E. Gonzalez, and I. Stoica, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023.
- [31] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in Proc. USENIX Security Symp., Vancouver, Canada, 2021, pp. 2633–2650.
- [32] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," Found. Trends Theor. Comput. Sci., vol. 9, no. 3–4, pp. 211–407, 2014.
- [33] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. NeurIPS, Long Beach, CA, USA, 2017, pp. 4765–4774.
- [34] Y. Luo, Z. Zhang, X. Zhou, C. Sun, Y. Bao, Y. Hu, D. Guo, S. Chen, and J. Wang, "NVIDIA A100 tensor core GPU: Performance and innovation," IEEE Micro, vol. 41, no. 2, pp. 29–35, Mar. 2021.
- [35] J. Li, H. Zheng, Y. Liu, X. Zhang, and Y. Chen, "Deep learning based resource allocation for AI inference workloads: A comparative study," IEEE Trans. Netw. Serv. Manag., vol. 20, no. 1, pp. 289–302, Mar. 2023.
- [36] X. Yao, Y. Xie, B. Zhao, and C. Zhang, "LLM agent memory management: Short-term context and long-term retrieval," in Proc. EMNLP Findings, Singapore, 2023, pp. 5234–5247.

- [37] S. Lee, K. Kim, S. Kim, J. Kim, D. Kim, H. Kim, and J. Kim, "HBM-PIM: An in-memory processing solution in high bandwidth memory for deep learning accelerators," *IEEE Micro*, vol. 41, no. 3, pp. 20–30, May–Jun. 2021.
- [38] A. Jain, A. K. K. Paul, and D. K. Panda, "Characterizing and optimizing GPU memory isolation for multi-tenant deep learning inference," in *Proc. IEEE CLUSTER*, Santa Fe, NM, USA, 2022, pp. 1–12.
- [39] M. Kang, Y. Lee, K. Jung, and J. Park, "Online learning for adaptive OS scheduling in dynamic AI workload environments," in *Proc. IEEE INFOCOM*, Vancouver, Canada, 2024, pp. 2781–2790.
- [40] M. Mollison, J. Erickson, J. Anderson, S. Baruah, and J. Scoredos, "Mixed-criticality real-time scheduling for multicore systems," in *Proc. IEEE ICSS*, Hangzhou, China, 2023, pp. 1–9.
- [41] A. Klimovic, H. Litz, and C. Kozyrakis, "Towards standardized interfaces for AI workload resource management," *ACM SIGOPS Oper. Syst. Rev.*, vol. 58, no. 1, pp. 12–24, Mar. 2024.
- [42] P. Sharma, Y. Zhao, K. Chaudhary, S. Gopalakrishnan, and H. Shah, "Disaggregated memory with CXL 3.0 for large-scale AI training," in *Proc. IEEE Hot Interconnects*, Santa Clara, CA, USA, 2023, pp. 1–8.